$$
\begin{array}{rl}
e ::= x \mid c \mid \text{true} \mid \text{false} \mid k_{\#n} \mid () & \text{constants} \\
\mid \textbf{let}\ x = e_1\ \textbf{in}\ e_2 & \text{let binding} \\
\mid \textbf{uniform}(e_1, e_2) \mid \textbf{gaussian}(e_1, e_2) \mid \textbf{exponential}(e) \mid \ldots & \text{continuous distributions} \\
\mid \textbf{discrete}(p_0, \ldots, p_{n-1}) & \text{discrete distribution} \\
\mid e_1 < e_2 \mid e_1 \le e_2 \mid e_1 <_{\#n} e_2 \mid e_1 \le_{\#n} e_2 & \text{comparisons} \\
\mid e_1\ \&\&\ e_2 \mid e_1 \mid\mid e_2 \mid \text{not}\ e & \text{Boolean operations} \\
\mid \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 & \text{conditional} \\
\mid (e_1, e_2) \mid \textbf{fst}\ e \mid \textbf{snd}\ e & \text{pairs} \\
\mid \textbf{fun}\ x\ \to\ e \mid e_1\ e_2 \mid \textbf{fix}\ f\ x := e & \text{functions} \\
\mid \textbf{observe}\ e \mid e_1; e_2 & \text{effects} \\
\mid \text{nil} \mid e_1 :: e_2 \mid (\textbf{match}\ e\ \textbf{with} \mid \text{nil} \to e_{\text{nil}} \mid e_1 :: e_2 \to e_{\text{cons}}\ \textbf{end}) & \text{lists} \\
\mid \textbf{diverge} & \text{diverge}
\end{array}
$$

Fig. 5. Syntax of the Slice language. Slice supports a variety of continuous distributions with one and two parameters; for a full list of supported distributions, see Appendix C.

We now turn to the more involved example in Figure 4, which Slice discretizes automatically. This example emphasizes how our discretization procedure operates across higher-order functions, unbounded recursion, and lists.

The program models the following process: We start by randomly sampling 4 intervals (l. 6 and 13), stored as a list of pairs, where the first component is a flag and the second component is a function that determines whether a given point falls within that interval. We then employ unbounded recursion (l. 36) to repeat the following a potentially unbounded number of times according to a geometric distribution: sample a point y from a gaussian (l. 31), and update the flag of every interval to true whenever y falls within that interval (l. 22). The program returns true iff *all* intervals have been hit at least once (l. 50 and 42). Exact inference on this program therefore determines the probability that this infinite stochastic process hits every interval at least once.

Slice *automatically* produces a *provably equivalent* (cf. §6) discrete program (shown in §A.5). Even though this program still contains unbounded recursion, our back-end automatically determines that its stochastic execution behavior can be modeled as a *finite-state* Markov chain. Our key insight is that exact inference can then be performed via *probabilistic model checking*: we use Storm [16], which automatically computes that the sought-after probability is 0.034474.... See §7.1 for details. To the best of our knowledge, our pipeline is the first capable of performing exact inference on suitable programs containing continuous sampling, higher-order functions, and unbounded recursion. In §7.1.2, we extend this program by conditioning *inside the recursion*.

3 The Slice Language and Type System

We now describe the language (§3.1), its types (§3.2), and the typing rules for discretization (§3.3). Henceforth, we use the terms *expression*, *subexpression*, and *program* somewhat interchangeably when the intended meaning is clear from context.

3.1 The Slice Language

The syntax in Figure 5 extends a standard higher-order functional core with probabilistic choices. Beyond constants, variables, comparisons, Boolean operators, conditionals, pairs, and lists, Slice offers first-class functions, recursion, and observations. The construct **observe**(e) conditions on e holding. Finite-domain constants $k_{\#n}$ range over $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$ as discussed below. For random sampling, we include the usual scalar ordered discrete distributions and any continuous distribution with a continuous distributions with tractable cumulative distribution function functions (one- or two-argument variants such as exponential, laplace, uniform, gaussian, beta, gamma). Adding further distributions is straightforward as long as their CDFs are known; An atomic distribution additionally requires access to CDF left limits. §C lists the ones implemented distributions implemented by Slice.

```
(* Returns true if y is in [lo,hi) *)
let within = fun lo -> fun hi -> fun y ->
  (y >= lo) && (y < hi) in

(* Function for sampling interval bounds *)
let b = fun _ ->
  discrete(0.18: 0.05, 0.11: 0.24,
           0.21: 0.37, 0.09: 0.58,
           0.41: 0.89) in

(* Initialize intervals by sampling their
respective (possibly overlapping) bounds *)
let init =
  (false, within 1 (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) 8) ::
  nil in

(* Update flags depending on y falling
   within the respective interval *)
let update = fix update y := fun ys ->
  match ys with
    | nil -> nil
    | h :: t ->
        ((fst h || ((snd h) y)), snd h)
        :: (update y t)
  end in

(* Sample from a discretized gaussian,
    then use the resulting value to update
    flags *)
let step = fun ys ->
   update (discrete(0.5, 0.0199388,
                    0.0748961, 0.0494739,
                    0.0747339, 0.0632619,
                    0.0309625, 0.0280777,
                    0.158655)) ys in

 (* Number of samples determined by
    unbounded recursion *)
 let cover = fix cover st :=
   if discrete(0.1, 0.9) < 1 then st
   else cover (step st) in

(* Have all intervals been covered at
   least once? *)
let is_covered = fix is_covered ys :=
  match ys with
    | nil -> true
    | h :: t -> (fst h) && (is_covered t)
  end in

(* Run entire process on sampled intervals *)
let final = cover init in
  is_covered final
```

Fig. 20. A *discretized* infinite stochastic process for random interval covering over $[0, 1]$, involving continuous sampling, higher-order recursion, and lists.

B.2 Bloom Filter

The full program for the case study from §7.1.3 is shown in Figure 23. The discretized program is shown in Figure 24.

C List of Supported Slice Distributions

The list of distributions supported scalar ordered distributions implemented by Slice, along with their probability density / mass functions, is given in the following table. supports discrete distributions by applying the same cut-set and value-set analysis used for continuous distributions, *provided that the discrete distribution admits a well-defined CDF*. The extension to discrete distributions does not introduce any fundamental changes to the discretization algorithm. For a distribution with law μ, discretization assigns the canonical cell $\mathrm{Intervals}(B)_k$ the mass

$$p_k = \mu(\mathrm{Intervals}(B)_k).$$

Equivalently, for a right-continuous CDF $F(c) = \Pr[X \le c]$, a strict cut $< c$ has cumulative boundary mass $F(c^-) = \Pr[X < c]$, whereas a non-strict cut $\le c$ has cumulative boundary mass $F(c)$. This distinction preserves atoms, including singleton cells induced by the two cuts $< c$ and $\le c$. The implementation currently provides this interface for Poisson and binomial distributions. Other scalar ordered distributions require the same CDF and CDF-left-limit interface. Vector-valued distributions such as multinomial require a different partition abstraction and are not supported by the real-line cut analysis.

This atomic-distribution interface is an implementation extension. The formal discretization and soundness arguments in this paper analyze the strict uniform family; they do not by themselves prove semantic preservation for the Poisson or binomial implementation paths. Extending the theorem requires an additional sampling case based on the tagged cumulative masses above.